

T.C. TEKNOFEST HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

MERGEN PROJESİ KRİTİK TASARIM RAPORU

Arayüz Yazılımı, Algoritma Analizi ve Maliyet Raporu

Bu rapor, Mergen projesinin arayüz yazılımı, görüntü işleme veri akışı, takip algoritması, haberleşme protokolü, yazılımsal güvenlik katmanları ve yazılım maliyetleri için hazırlanmıştır. Donanım ve mekanik tasarım kalemleri kapsam dışı bırakılmıştır.

Parametre	Değer
Arayüz altyapısı	Python ve PyQt6, asenkron QThread mimarisi
Haberleşme	Teensy 4.1 ile USB-Serial UART, JSON paket yapısı veya özel byte array
Video kaynağı	HuskyLens akıllı kamera, 320x240 hedef koordinatı
Takip yaklaşımı	Lazer ayrı hedef olarak tanımlanmaz; hedef merkezi kamera merkezine sürülür
Kontrol döngüsü	Teensy üzerinde Kalman filtresi ve PID kontrol

BÖLÜM 1: ARAYÜZ YAZILIMI MİMARİSİ VE STATE YÖNETİMİ

1.1. MVC Yapısı ve Çoklu İş Parçacığı Mimarisi

Mergen arayüz yazılımı Python ve PyQt6 tabanlı asenkron QThread mimarisi üzerine tasarlanmıştır. Ana UI thread yalnızca widget çizimi, kullanıcı etkileşimi ve görüntü paneli güncellemesinden sorumludur. Serial veri okuma, paket ayrıştırma, komut kuyruğu, ACK takibi ve log yazma işlemleri ayrı worker thread yapıları ile yürütülür.

MVC Katmanı		Yazılımsal Sorumluluk
Model		Sistem modu, hedef koordinatı, bbox, lock durumu, PID hata değerleri, bağlantı ve hata state alanları
View		PyQt6 ana pencere, video paneli, bounding box çizimi, telemetri göstergeleri, alarm ve kalibrasyon panelleri
Controller		USB-Serial haberleşme, komut gönderme, ACK takibi, mod geçişleri, fail-safe kararları
Thread	Sınıf	Görev
Main UI Thread	MainWindow	PyQt6 widget güncelleme, video overlay, kullanıcı komutları
Serial Worker	SerialWorker(QThread)	USB-Serial veri okuma, JSON/byte array parse, paket validasyonu
Vision State Worker	VisionWorker(QThread)	HuskyLens hedef metadata işleme, bbox ve lock state üretimi
Command Worker	CommandWorker(QThread)	Komut kuyruğu, sequence numarası, ACK timeout kontrolü
Logger Worker	LoggerWorker(QThread)	CSV, TXT ve JSONL kara kutu kayıtları

Worker thread yapıları UI bileşenlerine doğrudan erişmez. Veri aktarımı pyqtSignal ve pyqtSlot mekanizmasıyla yapılır. Bu tasarım, video render döngüsü sırasında serial okumanın bloklanmasını ve yüksek frekanslı telemetri sırasında arayüzün donmasını engeller.

```
class SerialWorker(QThread):
    telemetry_received = pyqtSignal(dict)
    ack_received = pyqtSignal(dict)
    connection_lost = pyqtSignal()
    packet_error = pyqtSignal(str)
```

```
serial_worker.telemetry_received.connect(main_window.on_telemetry_received)
serial_worker.connection_lost.connect(main_window.on_connection_lost)
```

1.2. HuskyLens - Teensy - Motor Kontrol Uçtan Uca Yazılımsal Akış Algoritması

Sistemde HuskyLens yalnızca hedef nesneyi takip eder. Lazer noktası ayrı bir görüntü hedefi olarak eğitilmez. Lazerin optik ekseninin kamera merkezi ile mekanik olarak hizalı olduğu kabul edilir. Kontrol hatası, hedef merkezi ile kamera merkezi arasındaki farktır.

```
INITIALIZE:
    ui_state = POWER_OFF
    tracking_state = NO_TARGET
    serial_port = CLOSED
    command_queue = EMPTY
    log_session = CREATE_NEW_SESSION()
```

```
SERIAL_WORKER_LOOP:
    while serial_port is open:
        raw_packet = serial.read_until('\n')
        packet = parse_json_or_custom_byte_array(raw_packet)
        if packet.type == TELEMETRY:
            emit telemetry_received(packet)
        if packet.type == ACK:
            emit ack_received(packet)
```

```
MAIN_UI_ON_TELEMETRY(packet):
    update_azimuth_actual, elevation_actual, roll, pitch, yaw, rssi
    if packet.target_detected == TRUE:
        Xt = packet.target_cx
        Yt = packet.target_cy
        ex = Xt - camera_center_x - laser_offset_x
        ey = Yt - camera_center_y - laser_offset_y
        tracking_state = TRACKING
        draw_bounding_box(packet.bbox_x, packet.bbox_y, packet.bbox_w, packet.bbox_h)
    else:
        lost_frame_count += 1
        if lost_frame_count <= 5: tracking_state = HOLD_LAST
        if lost_frame_count > 20: tracking_state = SEARCHING
```

```
FAILSAFE_LOOP:
  if telemetry_timeout > 500 ms: show yellow warning
  if telemetry_timeout > 1500 ms: send SAFE_HOLD
  if telemetry_timeout > 3000 ms: send LASER_OFF
```

BÖLÜM 2: GÖRÜNTÜ TABANLI TAKİP ALGORİTMASI VE HATA ANALİZİ

2.1. Piksel-Açı Dönüşüm Matematiği

HuskyLens çözünürlüğü 320x240 piksel olarak alınmıştır. Görüntü merkezi Cx=160 ve Cy=120 değerleridir.

```
W = 320, H = 240
Cx = W / 2 = 160
Cy = H / 2 = 120
ex = Xt - Cx - 0x
ey = Yt - Cy - 0y
```

Tasarım hesabında yatay görüş açısı FOVx=65 derece, dikey görüş açısı FOVy=50 derece kabul edilmiştir. Pinhole kamera modeline göre piksel cinsinden odak uzunlukları aşağıdaki şekilde hesaplanır.

```
fx = (W / 2) / tan(FOVx / 2) = 160 / tan(32.5 deg) = 251.14 px
fy = (H / 2) / tan(FOVy / 2) = 120 / tan(25.0 deg) = 257.34 px
theta_az = atan(ex / fx)
theta_el = atan(ey / fy)
theta_az_deg = theta_az * 180 / pi
theta_el_deg = theta_el * 180 / pi
```

Hesap	Sonuç
Yatay piksel başına açı	65 / 320 = 0.203125 derece/piksel
Dikey piksel başına açı	50 / 240 = 0.208333 derece/piksel
20 px yatay hata	atan(20 / 251.14) = 4.56 derece
20 px dikey hata	atan(20 / 257.34) = 4.45 derece
5 px hassas lock yatay hata	atan(5 / 251.14) = 1.14 derece
5 px hassas lock dikey hata	atan(5 / 257.34) = 1.11 derece

2.2. Kalman Filtresi ve PID Entegrasyonu

HuskyLens hedef koordinatlarında ışık değişimi, hedef kenarı titreşimi ve sınıflandırma kararsızlığı nedeniyle ölçüm gürültüsü oluşur. Teensy 4.1 üzerinde X ve Y koordinatları için bağımsız tek boyutlu Kalman filtreleri çalıştırılır.

Prediction:

```
x_hat[k|k-1] = x_hat[k-1|k-1]
P[k|k-1] = P[k-1|k-1] + Q
```

Correction:

```
K[k] = P[k|k-1] / (P[k|k-1] + R)
x_hat[k|k] = x_hat[k|k-1] + K[k] * (z[k] - x_hat[k|k-1])
P[k|k] = (1 - K[k]) * P[k|k-1]
```

Parametre	Anlam	Önerilen başlangıç
Q	Süreç gürültüsü	0.01
R	Ölçüm gürültüsü	2.00
P	Başlangıç belirsizliği	1.00

PID kontrol, filtrelenmiş açısal hatayı sıfıra yaklaştırmak için kullanılır.

$u[k] = K_p * e[k] + K_i * \sum(e[i] * dt) + K_d * (e[k] - e[k-1]) / dt$

Eksen	Kp	Ki	Kd
Azimuth	0.85	0.02	0.12
Elevasyon	0.75	0.015	0.10
Koruma		Değer	
Integral clamp		-100 ile +100	
Minimum hata deadband		±1 px	
Hassas lock eşiği		5 px	
Demo lock eşiği		20 px	

Koruma	Değer
PID döngü frekansı	100 Hz
HuskyLens veri frekansı	30 FPS

2.3. Algoritma Karmaşıklığı

İşlem	Zaman Karmaşıklığı	Alan Karmaşıklığı
JSON veya byte array parse	$O(L)$, L paket uzunluğu	$O(L)$
State güncelleme	$O(1)$	$O(1)$
Tek hedef bbox çizimi	$O(1)$	$O(1)$
Lock kontrolü	$O(1)$	$O(1)$
Kalman filtresi	$O(1)$	$O(1)$
PID kontrol	$O(1)$	$O(1)$
320x240 video render	$O(W*H)$	$O(W*H)$

320x240 çözünürlükte tek frame 76.800 piksel içerir. 30 FPS için 2.304.000 piksel/saniye, 60 FPS için 4.608.000 piksel/saniye işlenir. HuskyLens görüntü işleme görevini kendi üzerinde yürüttüğü için arayüz tarafındaki ana yük frame render ve metadata çizimidir.

Darboğaz	Risk	Çözüm
Serial verinin UI thread içinde okunması	Arayüz donması	SerialWorker(QThread)
Log dosyasının UI thread içinde yazılması	Buton gecikmesi	LoggerWorker(QThread)
Paket kuyruğu büyümesi	RAM artışı	Maksimum kuyruk uzunluğu 100
ACK bekleme	Komut belirsizliği	250 ms ACK timeout ve tekrar politikası
Telemetri kaybı	Güvensiz kontrol	safe_hold, laser_off, emergency_stop

BÖLÜM 3: AR-GE YAZILIM, LİSANS VE BİLGİSAYAR KAYNAK MALİYETİ

3.1. Hesaplama Maliyeti ve Sistem Gereksinimleri

60 FPS için frame süresi 16.67 ms, 30 FPS için frame süresi 33.33 ms değerindedir. UI çevrimi bu sürenin altında tamamlanmalıdır.

İşlem		Süre hedefi
Serial paket okuma		< 1 ms
Paket parse		< 1 ms
State güncelleme		< 0.2 ms
UI güncelleme		< 5 ms
BBox çizimi		< 1 ms
Log kuyruğuna yazma		< 0.5 ms
Bileşen	Minimum	Önerilen
CPU	2 çekirdek, 2.0 GHz x86-64	4 çekirdek, 3.0 GHz x86-64
RAM	4 GB	8 GB
GPU	Entegre GPU	Intel Iris Xe veya NVIDIA MX450
VRAM	512 MB paylaşımlı	2 GB
Disk	1 GB boş alan	10 GB SSD boş alan
Ekran	1366x768, 60 Hz	1920x1080, 60 Hz
İşletim sistemi	Windows 10 64-bit veya Ubuntu 22.04	Windows 11 64-bit veya Ubuntu 22.04 LTS
Python	3.10	3.10
UART baudrate	115200 baud	921600 baud

420 byte ortalama telemetri paketi 30 Hz gönderilirse UART 8N1 çerçeveleme nedeniyle 126.000 bit/s bant genişliği gerekir. 115200 baud bu yük için sınır altında kalır. Bu nedenle kritik telemetri paketi 256 byte altında tutulmalı veya 921600 baud kullanılmalıdır.

3.2. Yazılım ve Lisanslama BOM Tablosu

Yazılım	Görev	Lisans	Maliyet	Kısıtlama
Python 3.10	Ana programlama dili	PSF	0 TL	Ticari kullanıma uygun
PyQt6	GUI framework	GPL v3 veya ticari	GPL 0 TL, ticari lisans ücretli	Kapalı kaynak dağıtımda ticari lisans gerekir
Qt 6	PyQt6 altyapısı	GPL/LGPL/ticari	0 TL açık kaynak	Lisans koşullarına uyum zorunlu
pyserial	USB-Serial haberleşme	BSD	0 TL	Ticari kullanıma uygun
OpenCV	Video frame alma ve bbox çizimi	Apache 2.0	0 TL	Telif bildirimi korunmalı
NumPy	Sayısal hesaplama	BSD	0 TL	Ticari kullanıma uygun
ROS 2 Humble	Simülasyon ve ileri entegrasyon	Apache 2.0	0 TL	ROS paket lisansları korunmalı
CSV/TXT log altyapısı	Kara kutu kayıtları	Proje içi	0 TL	Veri bütünlüğü sağlanmalı
JSON protokol	Paketleme	Proje içi	0 TL	CRC ve sequence kontrolü eklenmeli

BÖLÜM 4: TEKNİK ŞARTNAME UYUMLULUK VE RİSK ANALİZİ

4.1. Şartname Uyum Durumu ve Doğrulama Analizi

Bu bölümde daha önce risk olarak belirlenen arayüz güvenliği, haberleşme sürekliliği, kalibrasyon ve kara kutu kayıt maddelerinin Mergen arayüz yazılımında karşılanan durumları verilmiştir. Komut adları yazılım protokolündeki eylem isimlerini belirtir; operatör arayüzünde bu işlemler Türkçe açıklama, durum göstergesi ve renk kodlu alarm yapısı ile sunulur.

Şartname Maddesi	Önlenen Risk	Mergen'de Uygulanan Çözüm	Doğrulama / Kabul Kriteri
Manuel/Otomatik mod geçiş güvencesi	Otomatik takipten manuele geçiş sırasında PID integral terimi, son hedef koordinatı ve motor komut kuyruğunun kontrolsüz kalması engellenir.	Arayüzde otomatik ve manuel mod komutları ayrı state üzerinden yönetilir. Mod değişimi komut paketi sequence numarasıyla gönderilir, ACK gecikmesi takip edilir ve komut kaydı oluşturulur.	Mod geçiş komutu commands.jsonl kaydında sequence numarasıyla görülür. ACK geldiğinde gecikme değeri arayüzde ve kayıt dosyasında doğrulanır.
Yazılımsal acil durdurma	Acil durdurma komutunun onay penceresi nedeniyle gecikmesi engellenir.	E-STOP butonu tek tıkla emergency_stop komutunu gönderir. Komut sequence numarasıyla commands.jsonl dosyasına yazılır. Simülasyon modunda sistem state değeri EMERGENCY_STOP yapılır.	Butona basıldığında emergency_stop kaydı oluşur. ACK paketi gelirse ACK gecikmesi ölçülür; ACK gelmezse pending komut listesinde izlenebilir.
Haberleşme kopması fail-safe yönetimi	USB-Serial veya ROS bağlantısı koptuğunda eski telemetrinin güncel kabul edilmesi engellenir.	Arayüz telemetri zaman aşımını takip eder. 500 ms seviyesinde uyarı, 1500 ms seviyesinde safe_hold, 3000 ms seviyesinde laser_off komutu üretilir. Bağlantı durumu panelde anlık gösterilir.	Telemetri kesildiğinde fail-safe alanı normal durumdan çıkar. safe_hold ve laser_off komutları commands.jsonl dosyasında gerekçe parametresiyle kayıt altına alınır.
Kamera-lazer eksen kaçıklığı kalibrasyonu	Kamera merkezi hedefleyen lazer optik ekseninin mekanik ofset nedeniyle hedef dışında kalması engellenir.	Kalibrasyon paneli eklendi. Kamera merkez X/Y, lazer ofset X/Y ve lock piksel eşiği operatör tarafından ayarlanır. Hata hesabı hedef merkezi eksi kamera merkezi eksi lazer ofset olarak yapılır.	Kalibrasyon uygulandığında set_param komutu gönderilir. Parametreler config_snapshot.jsonl dosyasına zaman damgasıyla yazılır.

Şartname Maddesi	Önlenen Risk	Mergen'de Uygulanan Çözüm	Doğrulama / Kabul Kriteri
Loglama ve kara kutu altyapısı	Test sonrası hedef kaybı, haberleşme kesintisi, operatör komutu ve kalibrasyon değişikliğinin geriye dönük kanıtlanamaması engellenir.	Her çalıştırmada runs klasörü altında ayrı oturum klasörü oluşturulur. Telemetri, komut, olay ve konfigürasyon kayıtları JSONL formatında tutulur.	events.jsonl, commands.jsonl, telemetry.jsonl ve config_snapshot.jsonl dosyaları oturum klasöründe oluşur. Her kayıt zaman damgası içerir.

Mergen arayüzünde kullanılan fail-safe eşikleri ve yazılım reaksiyonları aşağıdaki şekilde uygulanmıştır.

Telemetri Kesinti Süresi	Güvenlik Durumu	Arayüz Reaksiyonu	Teensy'ye Gönderilecek Komut
> 500 ms	Uyarı	Bağlantı göstergesi sarı renge alınır ve telemetri gecikmesi olay kaydına yazılır.	Komut gönderilmez.
> 1500 ms	Güvenli bekleme	Takip durumu SON KONUMDA BEKLE olarak gösterilir.	safe_hold: motor hedefi son geçerli konumda tutulur.
> 3000 ms	Kritik kayıp	Kırmızı alarm üretilir ve lazer durumu güvenli kapalı olarak işaretlenir.	laser_off: lazer çıkışı kapatılır.
> 5000 ms	Sistem güvensiz	Operatör müdahalesi zorunlu alarmı gösterilir.	emergency_stop: tüm hareket komutları iptal edilir.

Kara kutu telemetri kaydında kullanılan asgari alanlar aşağıdadır. Bu alanlar test sonrası kök neden analizi için tutulur.

timestamp_ms,seq,mode,tracking_state,target_detected,target_cx,target_cy,error_px_x,error_px_y,error_deg_az,error_deg_el,locked,az_actu

Kayıt Dosyası	İçerik	Doğrulama Amacı
telemetry.jsonl	Hedef koordinatı, açı değeri, lock durumu, RSSI ve hata bitleri	Takip kaybı ve kontrol performansı analizi
commands.jsonl	Gönderilen komut, sequence numarası, ACK durumu ve ACK gecikmesi	Komutun Teensy tarafından alınıp alınmadığını kanıtlama
events.jsonl	Mod geçişleri, fail-safe uyarıları ve operatör işlemleri	Operasyon zaman çizelgesi çıkarma
config_snapshot.jsonl	Kalibrasyon ve eşik parametreleri	Test tekrar edilebilirliği

SONUÇ

Mergen arayüz yazılımı, PyQt6 tabanlı asenkron QThread mimarisi ile gerçek zamanlı telemetri, hedef takip durumu, bounding box gösterimi, komut yönetimi, fail-safe reaksiyonları ve kara kutu kayıt altyapısını destekleyecek şekilde tasarlanmıştır. HuskyLens yalnızca hedef nesneyi tanır; lazer noktası görüntü hedefi olarak tanımlanmaz. Takip hatası hedef merkezi ile kamera merkezi farkından hesaplanır. Piksel hata, FOV tabanlı pinhole kamera modeliyle açı hatasına dönüştürülür. Teensy 4.1 üzerinde Kalman filtresi ve PID kontrol döngüsü çalışır. Arayüz UI thread'i bloklanmaz; serial, log ve komut işlemleri worker thread içinde yürütülür. Haberleşme kopması için safe_hold, laser_off ve emergency_stop güvenlik katmanları tanımlanır.